

Exercise 1: Parameterized Tests

EvenChecker.java:

```
package com.Jtesting;

public class EvenChecker {

    public boolean isEven(int number) {
        return number % 2 == 0;
    }
}
```

EvenCheckerTest.java:

```
package com.Jtesting;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;

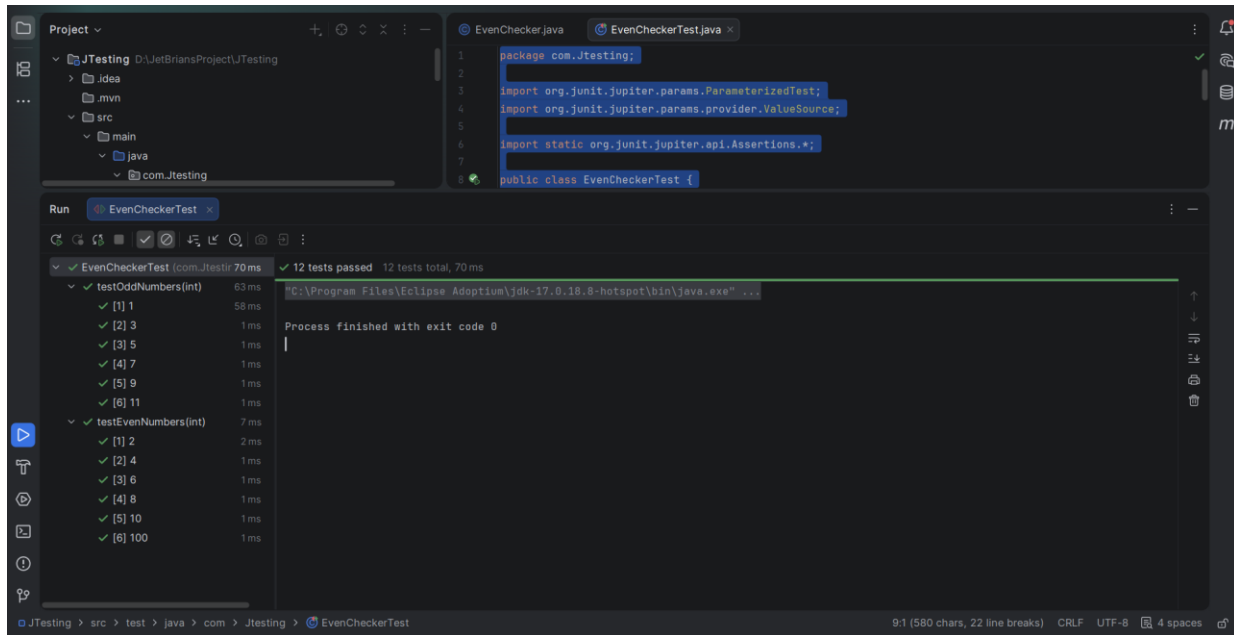
public class EvenCheckerTest {

    private final EvenChecker checker = new EvenChecker();

    @ParameterizedTest
    @ValueSource(ints = {2,4,6,8,10,100})
    void testEvenNumbers(int number) {
        assertTrue(checker.isEven(number));
    }

    @ParameterizedTest
    @ValueSource(ints = {1,3,5,7,9,11})
    void testOddNumbers(int number) {
        assertFalse(checker.isEven(number));
    }
}
```

Output:



Exercise 2: Test Suites and Categories

EvenCheckerBasicTest.java

```
package com.Jtesting;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class EvenCheckerBasicTest {

    EvenChecker checker = new EvenChecker();

    @Test
    void testEvenNumber() {
        assertTrue(checker.isEven(10));
    }

    @Test
    void testOddNumber() {
        assertFalse(checker.isEven(15));
    }
}
```

EvenCheckerParameterizedTest.java

```
package com.Jtesting;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
```

```
import static org.junit.jupiter.api.Assertions.*;

public class EvenCheckerParameterizedTest {

    EvenChecker checker = new EvenChecker();

    @ParameterizedTest
    @ValueSource(ints = {2, 4, 6, 8, 10})
    void testEvenNumbers(int number) {
        assertTrue(checker.isEven(number));
    }
}
```

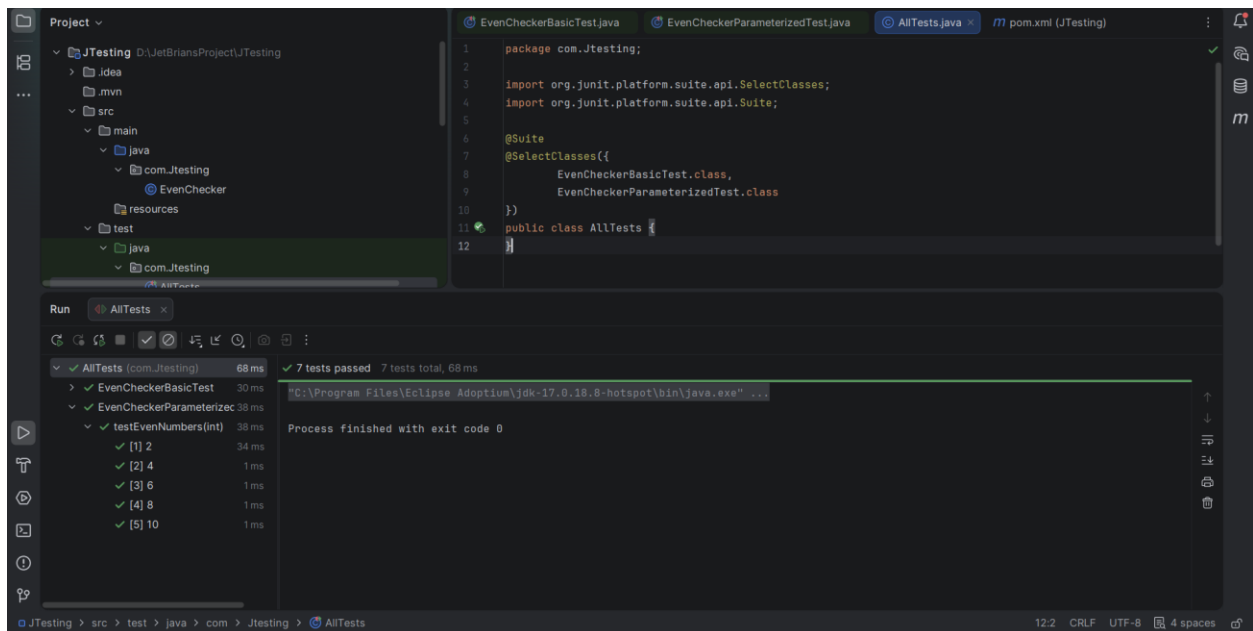
AllTests.java

```
package com.Jtesting;

import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({
    EvenCheckerBasicTest.class,
    EvenCheckerParameterizedTest.class
})
public class AllTests {
}
```

Output:



The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with folders for `idea`, `mvn`, `src` (containing `main` and `test`), and `resources`. The `test` folder contains a `java` sub-folder with the `com.Jtesting` package, which includes `EvenChecker` and `AllTests`.
- Editor:** Displays the source code of `AllTests.java`, which is a JUnit 5 suite class that includes `EvenCheckerBasicTest.class` and `EvenCheckerParameterizedTest.class`.
- Run Panel:** Shows the execution results of the `AllTests` suite. The output indicates that 7 tests passed in 68 ms. The test results are as follows:

Test Class	Test Method	Status	Time
EvenCheckerBasicTest	testEvenNumbers(2)	Passed	34 ms
EvenCheckerBasicTest	testEvenNumbers(4)	Passed	1 ms
EvenCheckerBasicTest	testEvenNumbers(6)	Passed	1 ms
EvenCheckerBasicTest	testEvenNumbers(8)	Passed	1 ms
EvenCheckerBasicTest	testEvenNumbers(10)	Passed	1 ms
EvenCheckerParameterizedTest	testEvenNumbers(2)	Passed	34 ms
EvenCheckerParameterizedTest	testEvenNumbers(4)	Passed	1 ms
EvenCheckerParameterizedTest	testEvenNumbers(6)	Passed	1 ms
EvenCheckerParameterizedTest	testEvenNumbers(8)	Passed	1 ms
EvenCheckerParameterizedTest	testEvenNumbers(10)	Passed	1 ms

Exercise 3: Test Execution Order

EvenChecker.java

```
package com.Jtesting;

public class EvenChecker {

    public boolean isEven(int number) {
        return number % 2 == 0;
    }
}
```

OrderedTest.java

```
package com.Jtesting;

import org.junit.jupiter.api.MethodOrderer;
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;

import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests {

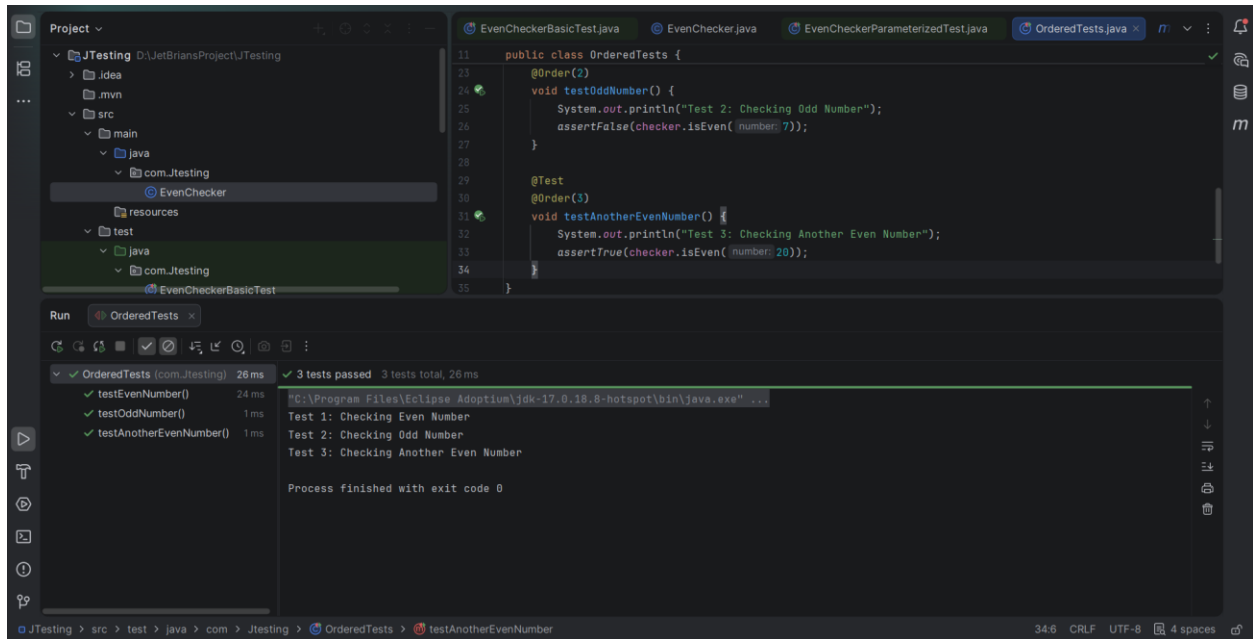
    EvenChecker checker = new EvenChecker();

    @Test
    @Order(1)
    void testEvenNumber() {
        System.out.println("Test 1: Checking Even Number");
        assertTrue(checker.isEven(10));
    }

    @Test
    @Order(2)
    void testOddNumber() {
        System.out.println("Test 2: Checking Odd Number");
        assertFalse(checker.isEven(7));
    }

    @Test
    @Order(3)
    void testAnotherEvenNumber() {
        System.out.println("Test 3: Checking Another Even Number");
        assertTrue(checker.isEven(20));
    }
}
```

Output:



Exercise 4: Exception Testing

EvenChecker.java

```
package com.Jtesting;

public class EvenChecker {

    public boolean isEven(int number) {
        return number % 2 == 0;
    }

    public void throwException(int number) {
        if (number < 0) {
            throw new IllegalArgumentException("Number cannot be negative");
        }
    }
}
```

ExceptionThrower.java

```
package com.Jtesting;

public class ExceptionThrower {

    public void throwException() {
        throw new IllegalArgumentException("Invalid Input");
    }
}
```

ExceptionThrowerTest.java

```
package com.Jtesting;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

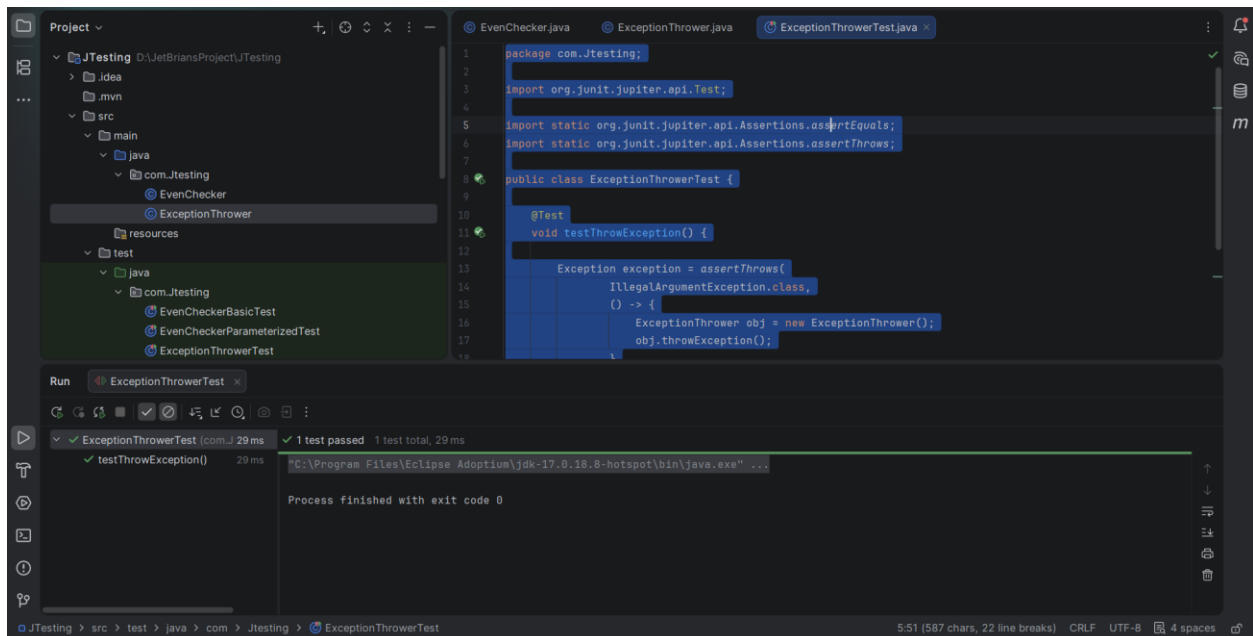
public class ExceptionThrowerTest {

    @Test
    void testThrowException() {

        Exception exception = assertThrows(
            IllegalArgumentException.class,
            () -> {
                ExceptionThrower obj = new ExceptionThrower();
                obj.throwException();
            }
        );

        assertEquals("Invalid Input", exception.getMessage());
    }
}
```

Output:



Exercise 5: Timeout and Performance Testing

PerformanceTester.java

```
package com.Jtesting;

public class PerformanceTester {

    public void performTask() {

        // Simulate some work
        for (int i = 1; i <= 1000000; i++) {
            Math.sqrt(i);
        }
    }
}
```

PerformanceTesterTest.java

```
package com.Jtesting;

import org.junit.jupiter.api.Test;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.assertTimeout;

public class PerformanceTesterTest {

    @Test
    void testPerformTask() {

        PerformanceTester tester = new PerformanceTester();

        assertTimeout(Duration.ofSeconds(2), () -> {
            tester.performTask();
        });
    }
}
```

Output:

The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project structure with a package `com.Jtesting` containing a class `PerformanceTesterTest`. The `test` directory is expanded, showing the `com.Jtesting` package.
- Code Editor:** Displays the source code of `PerformanceTesterTest.java`. The code includes imports for `org.junit.jupiter.api.Test` and `java.time.Duration`, and a static import for `org.junit.jupiter.api.Assertions.assertTimeout`. The class `PerformanceTesterTest` contains a single test method `testPerformTask()` that creates a `PerformanceTester` instance and asserts that `performTask()` completes within a 1-second timeout.
- Run Console:** Shows the execution of the test. The output indicates that the test `testPerformTask()` passed successfully, taking 29 ms. The console also shows the command used to run the test: `"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" ...` and the message "Process finished with exit code 0".

```
1 package com.Jtesting;
2
3 import org.junit.jupiter.api.Test;
4
5 import java.time.Duration;
6
7 import static org.junit.jupiter.api.Assertions.assertTimeout;
8
9 public class PerformanceTesterTest {
10
11     @Test
12     void testPerformTask() {
13
14         PerformanceTester tester = new PerformanceTester();
15
16         assertTimeout(Duration.ofSeconds(1), () -> {
17             tester.performTask();
18         });
19     }
20 }
```

Run PerformanceTesterTest

✓ PerformanceTesterTest (com., 29 ms) ✓ 1 test passed 1 test total, 29 ms

✓ testPerformTask() 29 ms

"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" ...

Process finished with exit code 0

JTesting > src > test > java > com > Jtesting > PerformanceTesterTest 4:1 (395 chars, 19 line breaks) CRLF UTF-8 4 spaces